

# MANUEL D'APPRENTISSAGE ACTIF & DE MAÎTRISE PRATIQUE

## Conception et Développement d'API Backend Hautement Robustes (Django 5 / DRF / Multi-Tenant)

Un parcours d'ingénierie cognitive et d'application métier intégrale fondé sur les sciences de l'apprentissage (*How Learning Works*, 2nd Ed. 2023) et les 4 grands piliers du raisonnement universel (Kahneman, Minto, Pólya, Meadows).

|                           |                                                                                                           |
|---------------------------|-----------------------------------------------------------------------------------------------------------|
| Stack Technologique :     | Python 3.12+   Django 5.x   Django REST Framework (DRF)   PostgreSQL Multi-tenant                         |
| Architecture Fondatrice : | Clean Architecture Django (Services & Selectors)   Isolation par Schéma Tenant                            |
| Méthodologie Cognitive :  | Règle des 70/30 (Apprentissage Actif)   Diagnostic Système 1   Décomposition MECE   Simulation Pre-Mortem |
| Public & Finalité :       | Ingénieurs & Développeurs Backend CCD Digital — Du Concept Théorique au Déploiement Production            |
| Version & Date :          | Édition 1.0 — Septembre 2026                                                                              |

### ■ RÉFLEXION MÉTACOGNITIVE & LOOKING BACK (Pólya) : CONTRAT D'APPRENTISSAGE ACTIF (LA RÈGLE DES 70/30)

Ce document n'est pas un tutoriel passif. Si vous vous contentez de lire ce manuel comme un roman, vous subirez l'**illusion de compétence** (*Fluency Illusion*) : tout vous semblera limpide à la lecture, mais vous resterez bloqué dès qu'il faudra concevoir une nouvelle API de zéro face à une page blanche en entreprise.

Ce manuel a été rigoureusement conçu selon le cycle des **skills d'apprentissage agentique**. Chaque module démarre par un **cadre SCQA** et un **dé-biaisage de vos intuitions**, vous expose l'architecture experte en **arborescence MECE**, décortique les **compétences composantes**, vous confronte à une **simulation de crash Pre-Mortem**, et se clôture par un **défi actif de mise en pratique**.

# MODULE 1 : Cadrage Stratégique, Architecture & Prérequis

## Phase 1 du Cycle : Diagnostic des Prérequis & Changement Conceptuel (Kahneman / Minto)

### ■ CADRAGE SCQA (Minto) : POURQUOI L'ARCHITECTURE CLASSIQUE ÉCHOUE

**S (Situation)** : L'entreprise SOUMAFE SARL développe CCD Digital, une plateforme SaaS de gestion de chantier multi-tenant destinée au secteur BTP.

**C (Complication)** : Les tutoriels Django classiques préconisent des « Fat Models » (toute la logique dans `save()`) et des « Fat Views » monolithiques. En production multi-tenant, cette approche provoque des failles d'étanchéité de données, des requêtes N+1 catastrophiques et une dette technique ingérable.

**Q (Question Fondamentale)** : Comment structurer le backend pour que chaque endpoint soit 100% sécurisé, traçable, modulaire et testable de façon totalement étanche ?

**A (Réponse Architecturale)** : L'adoption stricte du patron **Clean Architecture (Services & Selectors)**, couplé au **Socle Commun** (modèle de base unifié, soft-delete, et permissions cumulatives à 2 niveaux).

### ■ ■ PIÈGE SYSTÈME 1 & WYSIATI (Kahneman) : AUDIT DES FAUSSES CONCEPTIONS INTUITIVES

**Idée fausse fréquente n°1 (Système 1)** : « *Les ModelViewSets de DRF sont plus rapides à coder, utilisons-les partout !* »

**Réalité cognitive d'entreprise** : Les ViewSets magiques cachent la logique métier dans des méthodes génériques. Dès que les règles BTP se complexifient (ex: validation d'un rapport par un tiers extérieur ou contrôle d'effectif), le ViewSet devient un monstre illisible et impossible à maintenir.

**Idée fausse fréquente n°2 (WYSIATI)** : « *obj.delete() supprime la ligne en base de données.* »

**Réalité du Socle Commun (§3.1)** : La suppression physique est formellement INTERDITE. Tout appel à `delete()` doit être détourné en suppression logique (*Soft Delete* avec horodatage UTC et utilisateur responsable).

## L'Arborescence MECE d'une Application Backend CCD Digital

Chaque module métier (ex: `apps/chantier/`) est rigoureusement découpé en 10 composants fonctionnels disjoints (norme MECE de Barbara Minto) :

| Couche / Dossier      | Responsabilité Unique                                                       | Interdiction Formelle (Garde-fous)                                 |
|-----------------------|-----------------------------------------------------------------------------|--------------------------------------------------------------------|
| <b>models/</b>        | Structure de données, types, relations, héritage <code>ModelBase</code> .   | Zéro logique métier complexe. Pas de suppression physique.         |
| <b>selectors/</b>     | Lecture pure (Read-Only). Optimisation SQL ( <code>select_related</code> ). | AUCUNE écriture, modification ou suppression en base.              |
| <b>services/</b>      | Écriture métier (Write). Règles de gestion, transactions atomiques.         | Ne manipule pas les requêtes HTTP (request).                       |
| <b>serializers/</b>   | Validation des entrées HTTP & transformation JSON sortant.                  | Ne prend pas de décisions métier lourdes (délègue aux services).   |
| <b>permissions.py</b> | Contrôle d'accès à 2 niveaux : Rôle Global + Assignment Projet.             | Un rôle global seul ne donne JAMAIS accès à un chantier.           |
| <b>views/</b>         | Contrôleur HTTP mince : dispatch verbes, codes HTTP, Swagger.               | Ne fait aucun calcul métier ni requête SQL directe.                |
| <b>urls.py</b>        | Routage des endpoints branchés sous <code>/api/v1/</code> dans le tenant.   | Ne pas oublier d'inclure dans <code>config/urls_tenant.py</code> . |
| <b>tests/</b>         | Tests d'intégration Pytest avec client multi-tenant authentifié.            | Ne jamais merger sans 100% des tests au vert.                      |

## MODULE 2 : La Persistance & Le Socle Commun (Modèles)

### Phase 2 & 3 du Cycle : Décomposition aux Invariants de George Pólya & Échafaudage de Maîtrise

#### ■ DÉCOMPOSITION MECE (Minto / Pólya) : ANALYSE DES INVARIANTS STRUCTURELS (PÓLYA)

Face à l'exigence métier de concevoir un système de gestion des **Incidents de Chantier** (aléas de sécurité, pannes, retards), appliquons l'analyse des invariants de Pólya :

- **Données connues (\$D\$)** : Chaque incident est rattaché à un Projet obligatoire, éventuellement à un Lot spécifique, possède un déclarant, une gravité (Faible/Moyenne/Critique) et un statut.
- **Contraintes absolues (\$C\$)** : Pas de suppression physique (Socle §3.1), traçabilité de l'auteur (§2.4), dates en TIMESTAMPTZ UTC (§1.2), et clé primaire UUID générée côté client.
- **Inconnue à modéliser (\$x\$)** : La classe IncidentChantier qui hérite obligatoirement de ModeleBase.

#### ■ Fichier : apps/chantier/models/incident.py

```
import uuid
from django.db import models
from django.utils import timezone
from django.utils.translation import gettext_lazy as _
from apps.core.models import ModeleBase

class GraviteIncident(models.TextChoices):
    FAIBLE = "FAIBLE", _("Faible - Sans impact immédiat")
    MOYENNE = "MOYENNE", _("Moyenne - Ralentissement")
    CRITIQUE = "CRITIQUE", _("Critique - Arrêt des travaux / Danger")

class StatutIncident(models.TextChoices):
    DECLARE = "DECLARE", _("Déclaré")
    EN_TRAITEMENT = "EN_TRAITEMENT", _("En cours de traitement")
    RESOLU = "RESOLU", _("Résolu")
    CLOTURE = "CLOTURE", _("Clôture")

class IncidentChantier(ModeleBase):
    projet = models.ForeignKey("projets.Projet", on_delete=models.RESTRICT, related_name="incidents")
    lot = models.ForeignKey("projets.Lot", on_delete=models.RESTRICT, null=True, blank=True, related_name="incidents")
    titre = models.CharField(_("titre"), max_length=200)
    description = models.TextField(_("description"))
    gravite = models.CharField(_("gravité"), max_length=20, choices=GraviteIncident.choices, default=GraviteIncident.MOYENNE, db_index=True)
    statut = models.CharField(_("statut"), max_length=20, choices=StatutIncident.choices, default=StatutIncident.DECLARE, db_index=True)
    date_survenance = models.DateTimeField(_("date"), default=timezone.now)
    declare_par = models.ForeignKey("accounts.Utilisateur", on_delete=models.RESTRICT, related_name="incidents_declares")
    resolu_le = models.DateTimeField(_("résolu le"), null=True, blank=True)
    action_corrective = models.TextField(_("action corrective"), blank=True, default="")

    class Meta:
        verbose_name = _("incident de chantier")
        ordering = ["-date_survenance", "-cree_le"]
        indexes = [models.Index(fields=["projet", "statut"])]
```

#### ■ PRE-MORTEM : SIMULATION DU CRASH (Kahneman) : POURQUOI LE CASCADE EST INTERDIT DANS LE SOCLE COMMUN

**Scénario de Crash** : Un développeur utilise on\_delete=models.CASCADE au lieu de models.RESTRICT sur la clé étrangère projet.

**Conséquence Mortelle en Production** : Si un utilisateur désactive ou nettoie un projet de test, la suppression physique en cascade supprime instantanément 18 mois d'historique d'incidents, de litiges juridiques et de rapports d'assurance de l'entreprise. **Règle** : **Toujours RESTRICT sur les relations vitales.**

#### ■ DÉFI ACTIF EN ZONE PROXIMALE (Ericsson / Bjork) : DÉFI ACTIF #1 : L'HÉRITAGE DU SOCLE

**Question de mise en pratique active :**

Pourquoi n'avons-nous pas défini de champ id, cree\_le ou cree\_par dans la classe IncidentChantier ?

*Indice métacognitif* : Consultez l'héritage de ModeleBase dans apps/core/models/\_\_init\_\_.py. Quels sont les deux managers par défaut fournis ?

## MODULE 3 (Partie 1) : Les Sélecteurs (selectors/)

### Phase 3 du Cycle : L'Art de la Lecture Pure & Élimination Définitive du N+1 Queries

### La Règle d'Or de la Couche de Sélection

Un sélecteur ne fait **qu'une seule chose** : exécuter des requêtes de lecture optimisées en base de données. Il ne mute JAMAIS l'état, n'écrit aucune ligne et ne reçoit jamais d'objet HTTP request. Il retourne toujours un QuerySet paresseux (*lazy*) afin de permettre à la couche vue de le filtrer ou de le paginer sans surcoût mémoire.

#### ■ Fichier : apps/chantier/selectors/incident.py

```
# apps/chantier/selectors/incident.py
from uuid import UUID
from django.db.models import QuerySet
from apps.chantier.models.incident import IncidentChantier

def incidents_liste(
    *,
    projet_id: UUID | str | None = None,
    statut: str | None = None,
    gravite: str | None = None,
    date_apres=None
) -> QuerySet[IncidentChantier]:
    """Retourne la liste des incidents avec jointures SQL préchauffées."""
    # Préchauffage des relations vitales en une seule requête SQL JOIN
    qs = IncidentChantier.objects.select_related("projet", "lot", "declare_par").all()

    if projet_id:
        qs = qs.filter(projet_id=projet_id)
    if statut:
        qs = qs.filter(statut=statut)
    if gravite:
        qs = qs.filter(gravite=gravite)
    if date_apres:
        qs = qs.filter(date_survenance__gte=date_apres)

    return qs.order_by("-date_survenance")

def incident_par_id(incident_id: UUID | str) -> IncidentChantier | None:
    """Récupère un incident unique ou renvoie None sans lever d'exception."""
    try:
        return IncidentChantier.objects.select_related("projet", "lot", "declare_par").get(id=incident_id)
    except (IncidentChantier.DoesNotExist, ValueError):
        return None
```

#### ■ DÉCOMPOSITION MECE (Minto / Pólya) : LA BIBLE DE L'OPTIMISATION SQL (N+1 QUERIES)

**select\_related (SQL JOIN) :** À utiliser pour les clés étrangères simples (ForeignKey, OneToOne). Exécute un INNER JOIN ou LEFT JOIN unique. Dans notre sélecteur ci-dessus, projet, lot et declare\_par sont rapatriés en 1 seule requête au lieu de 301 requêtes pour une liste de 100 incidents !

**prefetch\_related (SQL IN) :** À utiliser pour les relations plusieurs-à-plusieurs (ManyToManyField) ou inverses (reverse ForeignKey). Exécute une 2e requête avec WHERE id IN (...) et assemble les objets en mémoire Python.

#### ■ DÉFI ACTIF EN ZONE PROXIMALE (Ericsson / Bjork) : DÉFI ACTIF #2A : QUERYSETS PARESSEUX

##### Question d'auto-évaluation active :

Que se passerait-il si vous appeliez list(incidents\_liste()) à l'intérieur du sélecteur au lieu de retourner le QuerySet brut ?

*Réponse attendue :* Vous forceriez l'évaluation immédiate de la requête SQL en chargeant 100% des lignes en mémoire RAM, détruisant tout le bénéfice de la pagination ultérieure dans la vue !

## MODULE 3 (Partie 2) : Les Services (services/)

### Phase 3 & 4 : Le Sanctuaire Métier, Règles de Gestion & Transactions Atomiques

### La Règle d'Or de la Couche de Service

Toutes les mutations de la base de données (création, mise à jour, changement d'état, résolution, suppression logique) sont **strictement concentrées ici**. Chaque service est décoré de `@transaction.atomic` et applique les règles de gestion (RG) de l'entreprise avant d'enregistrer les données.

#### ■ Fichier : `apps/chantier/services/incident.py`

```
# apps/chantier/services/incident.py
from django.core.exceptions import ValidationError
from django.db import transaction
from django.utils import timezone
from apps.chantier.models.incident import IncidentChantier, GraviteIncident, StatutIncident

@transaction.atomic
def creer_incident(*, projet, titre, description, declare_par, gravite=GraviteIncident.MOYENNE, lot=None, date_survenance=None) -> IncidentChantier:
    """Crée un incident après validation stricte des règles métier."""
    # Règle RG-02 : Cohérence spatiale du chantier
    if lot is not None and lot.projet_id != projet.id:
        raise ValidationError({"lot": "Le lot sélectionné n'appartient pas au chantier spécifié."})

    incident = IncidentChantier(
        projet=projet, lot=lot, titre=titre.strip(), description=description.strip(),
        gravite=gravite, statut=StatutIncident.DECLARE,
        date_survenance=date_survenance or timezone.now(),
        declare_par=declare_par, cree_par=declare_par
    )
    incident.full_clean()
    incident.save()
    return incident

@transaction.atomic
def resoudre_incident(*, incident: IncidentChantier, action_corrective: str, utilisateur) -> IncidentChantier:
    """Résout un incident en exigeant la consignation d'une action corrective."""
    if not action_corrective.strip():
        raise ValidationError({"action_corrective": "Une action corrective documentée est obligatoire."})
    incident.statut = StatutIncident.RESOLU
    incident.action_corrective = action_corrective.strip()
    incident.resolu_le = timezone.now()
    incident.save(update_fields=["statut", "action_corrective", "resolu_le", "modifie_le"])
    return incident

@transaction.atomic
def supprimer_incident(*, incident: IncidentChantier, utilisateur) -> None:
    """Suppression logique sécurisée (Socle Commun §3.1)."""
    incident.delete(utilisateur=utilisateur)
```

#### ■ PRE-MORTEM : SIMULATION DU CRASH (Kahneman) : L'ABSENCE DE TRANSACTION ATOMIQUE ET CORRUPTION D'ÉTAT

**Scénario de Crash en Production** : Un service exécute deux écritures : 1) Enregistrer l'incident, 2) Mettre à jour les statistiques de sécurité du chantier. Le serveur subit une micro-coupure de courant entre les deux requêtes.

**Sans `@transaction.atomic`** : La première écriture est persistée, la seconde avorte. La base de données de l'entreprise est corrompue et les tableaux de bord affichent des bilans faux.

**Avec `@transaction.atomic`** : Le moteur PostgreSQL annule automatiquement la première écriture (Rollback). Aucune donnée corrompue ne peut subsister.

#### ■ DÉFI ACTIF EN ZONE PROXIMALE (Ericsson / Bjork) : DÉFI ACTIF #2B : CONCURRENCE ET UPDATE\_FIELDS

##### Défi d'écriture active :

Observez l'appel `incident.save(update_fields=[...])` dans `resoudre_incident`. Pourquoi est-il vital de préciser `update_fields` plutôt qu'un `incident.save()` global ?

*Réflexion attendue* : Éviter les écrasements concurrents (Race Conditions) ! Si un autre utilisateur modifiait la description au même instant, un `save()` complet écraserait sa modification.

## MODULE 4 : Sécurité & Permissions à Double Niveau

### Socle Commun §2.3 : Contrôle Cumulatif Inviolable (Rôle Global + Affectation Projet)

#### ■ DÉCOMPOSITION MECE (Minto / Pólya) : LE DOUBLE VERROU DE SÉCURITÉ DU SOCLE COMMUN

**Règle de Gestion Fondatrice (RG-01) :** Le rôle global seul ne donne accès à RIEN.

Pour agir sur une ressource de chantier, deux verrous doivent être franchis **simultanément** :

- **Niveau 1 (Rôle Global) :** L'utilisateur possède-t-il la qualification professionnelle requise (ex: Conducteur de Travaux, Chef de Chantier, Admin) ? Géré par `RoleRequis.pour(...)`.
- **Niveau 2 (Appartenance Projet) :** L'utilisateur fait-il partie de l'équipe officiellement affectée à CE chantier précis ? Géré par `MembreDuProjet()`.

Si un utilisateur tente de modifier l'identifiant du projet dans l'URL pour inspecter un chantier voisin, il reçoit immédiatement un code **403 FORBIDDEN**, sans aucune fuite de métadonnées.

#### ■ Fichier : `apps/chantier/permissions.py`

```
# apps/chantier/permissions.py
from rest_framework import permissions
from apps.core.enums import RoleGlobal
from apps.core.permissions import RoleRequis, MembreDuProjet

class PeutConsulterIncidents(permissions.BasePermission):
    """Lecture : l'utilisateur doit obligatoirement être affecté au projet."""
    def has_object_permission(self, request, view, obj) -> bool:
        return MembreDuProjet().has_object_permission(request, view, obj.projet)

class PeutGererIncidents(permissions.BasePermission):
    """Création / Mutation : Réservée au management technique de ce chantier."""
    ROLES_AUTORISEES = (
        RoleGlobal.DIRECTEUR_PROJET,
        RoleGlobal.CHEF_PROJET,
        RoleGlobal.CONDUCTEUR_TRAVAUX,
        RoleGlobal.CHEF_CHANTIER,
        RoleGlobal.ADMIN,
    )

    def has_permission(self, request, view) -> bool:
        # Niveau 1 : Rôle global valide
        return RoleRequis.pour(*self.ROLES_AUTORISEES)().has_permission(request, view)

    def has_object_permission(self, request, view, obj) -> bool:
        # Niveau 2 : Affectation au projet
        return MembreDuProjet().has_object_permission(request, view, obj.projet)
```

#### ■ DÉFI ACTIF EN ZONE PROXIMALE (Ericsson / Bjork) : DÉFI ACTIF #2 : AUDIT D'AUTORISATION

##### Exercice de Pentesting Mental :

Un utilisateur connecté a le rôle global de *Directeur Général (DG)*. Il appelle l'endpoint PATCH `/api/v1/incidents/<uuid>/` pour clôturer un incident.

1. Franchit-il le Niveau 1 défini dans `PeutGererIncidents` ci-dessus ?
2. Que se passe-t-il s'il n'est pas expressément listé dans la table `ProjetMembre` du chantier ciblé ?

*Réflexion attendue :* Observez la liste `ROLES_AUTORISEES`. Le rôle DG n'y figure pas ! Même le patron de l'entreprise ne peut pas altérer directement la conduite de travaux sans passer par les délégations formelles.

## MODULE 5 : Sérialiseurs DRF & Filtres de Recherche

### Spécialisation des DTO (Data Transfer Objects) & Découplage Entrée/Sortie

### Pourquoi un unique ModelSerializer est un Anti-Pattern en Entreprise

Un tableau de bord listant 100 incidents a besoin d'objets allégés. Une vue détaillée a besoin de tous les horodatages et de l'audit. Un formulaire de déclaration ne doit accepter que les champs modifiables par l'utilisateur. Nous créons donc systématiquement **4 sérialiseurs spécialisés** :

#### ■ Fichier : apps/chantier/serializers/incident.py

```
# apps/chantier/serializers/incident.py
from rest_framework import serializers
from apps.chantier.models.incident import IncidentChantier, GraviteIncident
from apps.projets.models import Lot, Projet

class IncidentListSerializer(serializers.ModelSerializer):
    """Sérialiseur allégé pour l'affichage en tableau de bord."""
    projet_nom = serializers.CharField(source="projet.nom", read_only=True)
    lot_libelle = serializers.CharField(source="lot.libelle", read_only=True, default=None)
    declare_par_nom = serializers.CharField(source="declare_par.nom_complet", read_only=True)

    class Meta:
        model = IncidentChantier
        fields = ["id", "titre", "gravite", "statut", "date_survenance", "projet_nom", "lot_libelle", "declare_par_nom", "resolu_le"]

class IncidentDetailSerializer(serializers.ModelSerializer):
    """Sérialiseur complet avec audit et action corrective."""
    class Meta:
        model = IncidentChantier
        fields = ["id", "titre", "description", "gravite", "statut", "date_survenance", "action_corrective", "resolu_le", "cre_le", "m"]

class IncidentCreateSerializer(serializers.Serializer):
    """Validation stricte de la charge utile (payload) entrante."""
    projet_id = serializers.PrimaryKeyRelatedField(queryset=Projet.objects.all(), source="projet")
    lot_id = serializers.PrimaryKeyRelatedField(queryset=Lot.objects.all(), source="lot", required=False, allow_null=True)
    titre = serializers.CharField(max_length=200)
    description = serializers.CharField()
    gravite = serializers.ChoiceField(choices=GraviteIncident.choices, default=GraviteIncident.MOYENNE)
    date_survenance = serializers.DateTimeField(required=False)

    def validate(self, attrs):
        if attrs.get("lot") and attrs["lot"].projet_id != attrs["projet"].id:
            raise serializers.ValidationError({"lot_id": "Ce lot n'appartient pas au chantier spécifié."})
        return attrs
```

#### ■ Fichier : apps/chantier/filters.py

```
# apps/chantier/filters.py
import django_filters
from apps.chantier.models.incident import IncidentChantier, GraviteIncident, StatutIncident

class IncidentChantierFilter(django_filters.FilterSet):
    projet = django_filters.UUIDFilter(field_name="projet_id")
    gravite = django_filters.ChoiceFilter(choices=GraviteIncident.choices)
    statut = django_filters.ChoiceFilter(choices=StatutIncident.choices)
    date_apres = django_filters.DateTimeFilter(field_name="date_survenance", lookup_expr="gte")

    class Meta:
        model = IncidentChantier
        fields = ["projet", "gravite", "statut", "date_apres"]
```

## MODULE 6 : Les Vues API REST, Pagination & Swagger

### Le Contrôleur HTTP Mince & la Documentation Automatisée OpenAPI (drf-spectacular)

#### ■ CADRAGE SCQA (Minto) : ANATOMIE D'UNE VUE API CONFORME

Règle de Conception d'une APIView Experte :

1. **Parser JSON explicite** : Toujours déclarer `parser_classes = [JSONParser]`.
2. **Pagination Standardisée** : Toujours brancher `PaginationStandard` sur les listes.
3. **Délégation Totale** : GET délègue aux selectors, POST/PATCH délèguent aux services.
4. **Documentation Intégrée** : Chaque méthode HTTP porte l'annotation `@extend_schema` pour alimenter automatiquement Swagger UI (`/api/v1/docs/`).

#### ■ Fichier : `apps/chantier/views/incident.py`

```
# apps/chantier/views/incident.py
from drf_spectacular.utils import extend_schema
from rest_framework import status
from rest_framework.parsers import JSONParser
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from rest_framework.views import APIView
from apps.chantier.filters import IncidentChantierFilter
from apps.chantier.permissions import PeutConsulterIncidents, PeutGererIncidents
from apps.chantier.selectors.incident import incident_par_id, incidents_liste
from apps.chantier.serializers.incident import IncidentCreateSerializer, IncidentDetailSerializer, IncidentListSerializer
from apps.chantier.services.incident import creer_incident, resoudre_incident, supprimer_incident
from apps.core.pagination import PaginationStandard

class IncidentListCreateView(APIView):
    parser_classes = [JSONParser]
    pagination_class = PaginationStandard

    def get_permissions(self):
        if self.request.method == "POST":
            return [IsAuthenticated(), PeutGererIncidents()]
        return [IsAuthenticated(), PeutConsulterIncidents()]

    @extend_schema(summary="Lister les incidents", responses={200: IncidentListSerializer(many=True)})
    def get(self, request):
        qs = incidents_liste()
        filtre = IncidentChantierFilter(request.GET, queryset=qs)
        paginator = self.pagination_class()
        page = paginator.paginate_queryset(filtre.qs, request)
        return paginator.get_paginated_response(IncidentListSerializer(page, many=True).data)

    @extend_schema(summary="Déclarer un incident", request=IncidentCreateSerializer, responses={201: IncidentDetailSerializer})
    def post(self, request):
        serializer = IncidentCreateSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        incident = creer_incident(declare_par=request.user, **serializer.validated_data)
        return Response(IncidentDetailSerializer(incident).data, status=status.HTTP_201_CREATED)
```

#### ■ Fichier : `apps/chantier/urls.py`

```
# apps/chantier/urls.py
from django.urls import path
from apps.chantier.views.incident import IncidentListCreateView

app_name = "chantier"
urlpatterns = [
    path("incidents/", IncidentListCreateView.as_view(), name="incident-liste-creer"),
]
# Ce fichier est automatiquement monté sous /api/v1/ dans config/urls_tenant.py :
# path("api/v1/", include("apps.chantier.urls"))
```



# MODULE 7 : Tests Automatisés (Pytest) & Définition of Done

## Validation Empirique de la Robustesse & Intégration Continue (DoD)

■ Fichier : apps/chantier/tests/test\_api\_incidents.py

```
# apps/chantier/tests/test_api_incidents.py
import pytest
from rest_framework import status
from apps.chantier.models.incident import IncidentChantier, GraviteIncident

@pytest.mark.django_db
class TestIncidentAPI:
    def test_creation_incident_nominale(self, client_authentifie, projet_test, conducteur_travaux):
        client_authentifie.force_authenticate(user=conducteur_travaux)
        payload = {
            "projet_id": str(projet_test.id),
            "titre": "Panne Grue G1",
            "description": "Coupure d'alimentation sur le treuil principal.",
            "gravite": GraviteIncident.CRITIQUE,
        }
        res = client_authentifie.post("/api/v1/incidents/", payload, format="json")
        assert res.status_code == status.HTTP_201_CREATED
        assert res.data["titre"] == "Panne Grue G1"
        assert IncidentChantier.objects.filter(id=res.data["id"]).exists()

    def test_rejet_si_non_membre_du_projet(self, client_authentifie, projet_autre_entreprise, conducteur_travaux):
        client_authentifie.force_authenticate(user=conducteur_travaux)
        payload = {"projet_id": str(projet_autre_entreprise.id), "titre": "Intrusion", "description": "Test"}
        res = client_authentifie.post("/api/v1/incidents/", payload, format="json")
        # La muraille de sécurité Niveau 2 doit immédiatement bloquer la requête
        assert res.status_code == status.HTTP_403_FORBIDDEN
```

## Checklist de Conformité Strict avant Mise en Production (DoD)

| Jalon de Contrôle        | Exigence Technique                                                 | Validation  |
|--------------------------|--------------------------------------------------------------------|-------------|
| 1. Héritage Socle        | Le modèle hérite de ModeleBase (UUID, dates UTC, audit).           | OBLIGATOIRE |
| 2. Zéro Delete Physique  | La suppression appelle delete(utilisateur=...) (Soft-delete).      | OBLIGATOIRE |
| 3. Optimisation SQL      | Le sélecteur utilise select_related (Zéro requête N+1).            | OBLIGATOIRE |
| 4. Atomicité Écriture    | Toute mutation dans services/ est décorée de @transaction.atomic.  | OBLIGATOIRE |
| 5. Sécurité 2 Niveaux    | La vue contrôle RoleGlobal ET MembreDuProjet.                      | OBLIGATOIRE |
| 6. Documentation Swagger | Chaque méthode porte @extend_schema (visible sur /api/v1/docs/).   | OBLIGATOIRE |
| 7. Qualité de Code       | Formatage et linter impeccables (ruff check . sans avertissement). | OBLIGATOIRE |
| 8. Suite de Tests        | Tests unitaires et intégration au vert (pytest à 100%).            | OBLIGATOIRE |

## MODULE 8 : Régulation Métacognitive & Looking Back

### Phase 5 du Cycle : Dé-biaisage Dunning-Kruger & Fiche d'Auto-Évaluation de Maîtrise

#### ■ RÉFLEXION MÉTACOGNITIVE & LOOKING BACK (Pólya) : L'ANALYSE RÉTROGRADE DE MAÎTRISE

##### L'Heuristique du 'Looking Back' de George Pólya :

La maîtrise ne s'arrête pas au moment où le code compile ou passe les tests. Prenez 3 minutes pour examiner la solution globale :

- 1. Généralisation** : Comment cette architecture (Modèle  $\rightarrow$  Sélecteur  $\rightarrow$  Service  $\rightarrow$  Sérialiseur  $\rightarrow$  Vue  $\rightarrow$  Test) s'applique-t-elle à une autre entité, par exemple la gestion du *Matériel de Chantier* ou des *Bons de Livraison* ?
- 2. Éléance & Simplicité** : Avez-vous laissé du code mort ou des calculs redondants ? Le service est-il pur et découplé de toute logique HTTP ?
- 3. Transfert Loïn** : Si l'entreprise décidait demain de migrer vers une API gRPC ou GraphQL, seule la couche views/ et serializers/ changerait ; vos services/ et selectors/ resteraient rigoureusement intacts. C'est l'essence même de la Clean Architecture.

### Votre Fiche Réflexive d'Auto-Évaluation (Learning Wrapper)

Avant de déclarer une tâche backend 'Terminée' en entreprise, répondez par écrit aux 4 questions ci-dessous :

| Question Métacognitive                                                                                                                   | Votre Auto-Diagnostic en Situation Réelle                                                                               |
|------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>1. Calibration de Confiance (0-100%)</b><br>Quel est votre degré de certitude que vos permissions bloquent un tiers non affecté ?     | [ ] % — Avez-vous testé explicitement le code HTTP 403 dans votre suite de tests ?                                      |
| <b>2. Audit de Charge Cognitive (Sweller)</b><br>Avez-vous décomposé votre logique en sous-fonctions limpides dans le service ?          | Le service contient-il plus de 40 lignes ? Si oui, isolez les sous-règles dans des fonctions privées.                   |
| <b>3. Recherche du Point Aveugle (WYSIATI)</b><br>Qu'avez-vous supposé comme acquis (ex: format de date, lot nul, utilisateur anonyme) ? | Vérifiez que votre sérialiseur gère explicitement allow_null=True et les chaînes vides.                                 |
| <b>4. Attitude Face à l'Erreur (Growth Mindset)</b><br>Si un bug survient en pré-production, comment réagissez-vous ?                    | L'erreur est une information scientifique précieuse. Écrivez immédiatement un test de régression qui reproduit l'échec. |

#### ■ DÉFI ACTIF EN ZONE PROXIMALE (Ericsson / Bjork) : LE MOT DU LEAD ARCHITECTE

##### Félicitations pour votre engagement dans ce cycle d'apprentissage !

En appliquant avec constance ces principes issus de *How Learning Works* et du *Deep Reasoning Engine*, vous ne développez pas seulement du code : vous bâtissez des systèmes pérennes et vous élevez votre pratique au rang d'**ingénierie logicielle d'élite**.